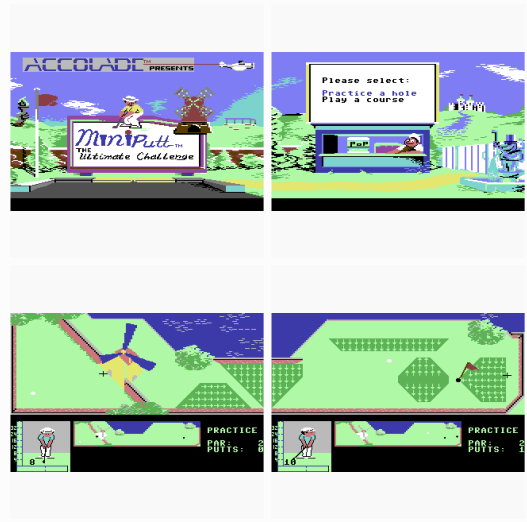


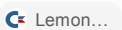


schau mal ob du weitere infos über dieses Spiel Findest
<https://www.lemon64.com/game/mini-putt> ich finde es sehr gut und es hat mir früher sehr viel Spaß gemacht. also will ich es für den PC neu schreiben. dafür benötige ich mehr infos.

54s nachgedacht

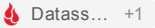
Ja. Ich habe noch einiges gefunden – darunter sogar das **Originalhandbuch**, **zeitgenössische Tests** und ein **vollständiges C64-Longplay aller vier Kurse**. Damit lässt sich das Spiel ziemlich gut rekonstruieren.

Was im Original tatsächlich steckt

Mini-Putt erschien 1987, wurde von **Artech Digital Entertainments** entwickelt und in den USA von Accolade veröffentlicht; in Europa über Electronic Arts. Für die C64-Version nennt Lemon64 **Lise Mendoza und Martin Edwards** als Programmierer, **Grant Campbell, Ross O'Conner und Steve Goodman** für die Grafik sowie **Paul Butler** für die Musik.  Lemon...

Das Spiel besitzt **vier vollständige Kurse mit jeweils neun Löchern** – also insgesamt **36 Bahnen**:

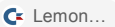
Kurs	Charakter
Traditional	klassisches Minigolf, besonders Bandenschüsse
Deluxe	stärkere Steigungen und kompliziertere Bahnen
Classic	verrückte, animierte Hindernisse
Challenge	schwierigster Kurs, komplexe Steigungen und Wege

Man kann entweder einen kompletten Kurs spielen oder gezielt **jedes einzelne Loch üben**. Bis zu vier Personen spielen nacheinander am selben Rechner. 

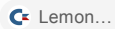
Das ist für deine Neuauflage schon eine schöne Größenordnung: **36 Level sind genug Inhalt, aber noch überschaubar**.

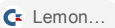
Das eigentliche Geheimnis des Spielgefühls

Das Schusssystem ist interessanter, als es auf den Screenshots zunächst aussieht.

Zuerst bewegt man ein Fadenkreuz über die Bahn. Neben Mini-Putt Pete wird dabei sogar die Entfernung zwischen Ball und Zielcursor angezeigt. Bei größeren Bahnen kann man den sichtbaren Bereich wechseln bzw. die Bahn erkunden. 

Danach folgt ein mehrstufiges Timing-System:

1. **Feuer drücken**: der Kraftbalken steigt.
2. Beim gewünschten Wert **Feuer erneut drücken** → die Schlagstärke wird festgelegt.
3. Danach läuft der Indikator zur **Accuracy-Bar**.
4. Dort muss möglichst exakt die Mittellinie getroffen werden.
5. Zu früh = Ball weicht nach links ab.
6. Zu spät = Ball weicht nach rechts ab.
7. Erst beim **Loslassen** wird der Ball tatsächlich gespielt. 

Gerade der letzte Punkt ist clever: Dadurch kann man einen fertig vorbereiteten Schlag vor einer **Windmühle oder einem anderen beweglichen Hindernis zurückhalten** und genau im richtigen Moment loslassen. Das Handbuch weist ausdrücklich auf diesen Trick hin. 

Das würde ich in einer PC-Version unbedingt erhalten. **Zielen + Kraft + kleine Ungenauigkeit + bewusstes Auslösen** erzeugt deutlich mehr Spiel als das heute übliche „Maus ziehen und loslassen“.

Die Physik ist erstaunlich systematisch

Das Handbuch beschreibt die Geländetypen ziemlich genau:

- **Smooth**: normales ebenes Grün.
- **Slopes**: Pfeile drücken den Ball in Pfeilrichtung; unterschiedliche Darstellung signalisiert unterschiedliche Stärke.
- **Walls**: Ball prallt abhängig von Richtung und Geschwindigkeit ab.
- **Water**: Ball wird an die Eintrittsstelle zurückgesetzt und es gibt einen Strafschlag.
- **Bridges**: können eben oder geneigt sein, offen oder geschlossen, beweglich oder ohne Geländer.

- **Loop Bridge:** verlangt beispielsweise eine bestimmte Mindestgeschwindigkeit.

 Lemon...

Für eine Neuprogrammierung ist das hervorragend, weil man daraus praktisch direkt ein einfaches Physiksystem ableiten kann:

```
Ball
├─ Position
├─ Geschwindigkeit
├─ Reibung
└─ Radius

Bahn
├─ normale Fläche
├─ Wände
├─ Gefälle-Zonen → zusätzlicher Kraftvektor
├─ Wasser-Zonen
├─ Loch
└─ Spezialobjekte
    ├─ Windmühle
    ├─ Brücke
    ├─ Tor
    ├─ Kanone
    └─ bewegliche Hindernisse
```

Man braucht dafür **keine komplizierte 3D-Golfphysik**. Das Original ist im Kern eine 2D-Simulation aus Kreisbewegung, Reibung, Reflexion und lokalen Kraftfeldern.

Der „Classic“-Kurs ist besonders interessant

Aus den erhaltenen C64-Screenshots lassen sich einige der verrückten Mechaniken identifizieren. MobyGames dokumentiert unter anderem:

bewegliches Tor → Elefantenrüssel → Windmühle → Flugzeugpropeller → Kanone → Schloss/Wasser → Kobra/Taj-Mahal → schwingender Baumstamm.  MobyGa...

Bei der Flugzeug-/Kanonenbahn muss der Ball beispielsweise in eine Kanone gelangen, die ihn anschließend automatisch in Richtung Loch schießt. Beim Schloss gibt es Wasser und eine bewegliche Zugbrücke.  MobyGam... +1

Das ist wichtig für das Remake: Die Hindernisse sind nicht bloß Kollisionsobjekte. Sie können **eigene kleine Spielregeln** haben.

Ich würde deshalb Spezialobjekte modular aufbauen:

```
Obstacle
  update()
  collide(ball)
  trigger(ball)
```

```
Windmill
  rotation_speed

Gate
  open_time
  closed_time

Cannon
  trigger_area
  output_position
  output_velocity

Bridge
  state
  animation
  collision_enabled
```

Dann kannst du später sehr einfach neue verrückte Bahnen bauen.

Mini-Putt Pete ist ebenfalls wichtiger als man denkt

Der kleine Golfer links unten ist keine reine Dekoration.

Zeitgenössische Rezensionen heben seine Reaktionen ausdrücklich hervor: Er freut sich über gute Schläge, steht frustriert da und **zerbricht sogar seinen Schläger**, wenn etwas entsprechend schlecht läuft bzw. der Ball ins Wasser geht.  Lemon64 +1

Das trägt offenbar stark zum Charme bei. Genau solche Kleinigkeiten würde ich in einer modernen Version **nicht durch ein steriles UI ersetzen**.

Im Gegenteil: Pete könnte sogar etwas stärker ausgebaut werden – ohne das Spiel komplizierter zu machen.

Interessant ist auch, was die damaligen Tester mochten

Die Zeitschrift *Zzap!64* gab dem Spiel **78 %**. Besonders hoch war „Hookability“ mit **82 %** – also wie schnell man hineinkommt und weiterspielen möchte. Kritik gab es vor allem am langsamen Nachladen der einzelnen Bahnen, an wenig Sound und daran, dass es nur vier Kurse gab.  Datass...

Compute! beschrieb genau den Effekt, an den du dich vermutlich Erinnerst: Das Spiel sehe zunächst unglaublich einfach aus, sei aber überraschend schwer. Hervorgehoben wurden die vier unterschiedlich schwierigen Kurse, die Gefälleanzeige, animierte Hindernisse und das einfache, aber präzisionsabhängige Schusssystem.  atarimagazines...

Das ist für ein Remake ein ziemlich gutes Designziel:

In zehn Sekunden verstanden – nach Stunden noch nicht gemeistert.

Und auf Lemon64 schreiben Spieler bis heute genau dasselbe: einfach zu spielen, überraschend fordernd und besonders im lokalen Mehrspieler sehr unterhaltsam.

 Lemon...

Eine sehr wertvolle Fundstelle: vollständiges C64-Longplay

Es gibt ein vollständiges PAL-C64-Longplay. Dort werden **alle vier Kurse komplett gespielt**:

- 00:40 – Deluxe
- 06:28 – Classic
- 13:05 – Traditional
- 18:36 – Challenge



Das ist wahrscheinlich unsere **wichtigste Referenz für die Rekonstruktion der 36 Bahnen**.

Auch das originale Handbuch ist vollständig als Text erhalten:

[Originales C64-Handbuch bei Lemon64](#)

Für eine PC-Neufassung würde ich das Original nicht „modernisieren“, sondern erweitern

Ich würde zunächst einen **sehr originalgetreuen Kern** bauen:

Version 0.1

Ballphysik
Wände
Reibung
Loch
Cursor-Zielen
Kraftanzeige
Accuracy-Anzeige
Gefällezonen

Version 0.2

9 Traditional-Bahnen
Score
Par
Übungsmodus
Hotseat 1-4 Spieler
Mini-Putt-Pete-Reaktionen

Version 0.3

bewegliche Hindernisse
Trigger
Kanonen
Brücken
Wasser
Classic Course

Danach Deluxe und Challenge.

In **Godot** passt das ausgesprochen gut: Eine Bahn könnte eine eigene Scene sein; Wände, Gefälle, Wasser und Hindernisse wären wiederverwendbare Nodes. Die eigentlichen Bahndaten könnten dadurch fast vollständig vom Programmcode getrennt bleiben.

Und ich sehe noch eine sehr interessante Erweiterung

Das Original hatte einen entscheidenden Kritikpunkt: **nur 36 Bahnen**.  atarimagazines... +1

Das könnten wir heute elegant lösen:

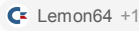
Course Editor → eigene Bahnen → Kurs aus 9 Bahnen zusammenstellen.

Dann wäre dein Spiel im Kern:

Mini-Putt 1987 + moderner Bahneditor + unbegrenzt neue Kurse

und nicht einfach nur ein grafisch aufgehübschtes Remake.

Ein rechtlicher Punkt ist dabei wichtig: Das Original ist trotz seines Alters nicht automatisch gemeinfrei. Das Handbuch bezeichnet Software und Spiel ausdrücklich als urheberrechtlich geschützt; Accolade wurde später übernommen, und Rechte an alten Accolade-Titeln sind durch mehrere Firmen gegangen. Welche Rechte **speziell an Mini-Putt** heute wo liegen, konnte ich bei dieser Suche noch nicht eindeutig belegen.

 Für ein eigenes kommerzielles Spiel würde ich deshalb **Mechanik und Spielgefühl übernehmen, aber Namen, Pete, Originalgrafiken und exakte Bahnen zunächst als Referenz behandeln.**

Ich würde als nächsten Schritt tatsächlich das Longplay durcharbeiten und daraus eine „**Mini-Putt Reverse-Engineering-Spezifikation**“ erstellen: alle 36 Löcher mit Par, Geometrie, Geländearten, Hindernissen, Bewegungen und Sonderregeln. Damit hätten wir anschließend praktisch den Bauplan für die PC-Version.

in der tat es sind einfach zu wenig Bahnen... was man machen kann sind 4 Unterschiedliche Spielerfiguren einbauen. einer dicht am original dann eine schwache Person somit ist die Kraft leichter zu dosieren aber das maximum ist nicht so hoch , kräftiger also schwerer zu dosieren kommt aber weiter und das dann auch im Ziehlen einmal weniger abweichung aber die mitte ist schwerer zu treffen und umgekehrt.

Ja, das passt hervorragend zu **Mini-Putt**, weil du damit Abwechslung bekommst, ohne die Grundsteuerung komplizierter zu machen. Der Spieler lernt weiterhin nur **Zielen** → **Kraft** → **Genauigkeit** → **Schlag**, aber dieselbe Bahn fühlt sich mit einer anderen Figur etwas anders an.

Ich würde die Charakterunterschiede dabei nicht als klassische RPG-Werte behandeln, sondern direkt an die beiden Anzeigen koppeln.

Vier Figuren könnten so funktionieren

Figur	Kraft	Kraft-Dosierung	Ziel-Mitte	Fehlerwirkung	Charakter
Pete / Klassiker	100 %	normal	normal	normal	Referenz
Feinfühlige	75 %	langsam	normal	normal	Kraft sehr gut dosier

Figur	Kraft	Kraft-Dosierung	Ziel-Mitte	Fehlerwirkung	Charakter
Kraftpaket	130 %	schnell	normal	normal	hohe Reichweite, schwierig zu dosier
Techniker	95 %	normal	schmal	geringe Abweichung	schwer perfekt zu treffen, Fehler wen schlimm

Gerade beim **Techniker** gefällt mir deine Idee sehr gut: Das perfekte Zentrum ist klein, aber wenn man danebenliegt, fliegt der Ball beispielsweise nur $\pm 4^\circ$ statt $\pm 8^\circ$ daneben.

Damit ist er nicht einfach „besser“. Ein guter Spieler kann mit Pete einen perfekten geraden Schlag schaffen. Beim Techniker ist der perfekte Schlag schwieriger, dafür sind mittelmäßige Schläge kontrollierbarer.

Man könnte sogar das Gegenprinzip für die schwächere Figur verwenden:

großes Zentrum, aber außerhalb davon starke Abweichung.

Also ungefähr:

Feinfühilige Figur



Das perfekte Fenster ist relativ groß.
Aber außerhalb davon steigt der Fehler schnell.

gegenüber:

Techniker



Perfekt ist schwer zu treffen.
Aber ein Fehlschlag ist weniger dramatisch.

Das ist spielerisch interessanter als einfach nur „Figur A hat Accuracy 8, Figur B Accuracy 6“.

Besonders schön ist die Wechselwirkung mit den Bahnen

Dann entstehen plötzlich natürliche Vor- und Nachteile.

Eine Bahn mit langem Anstieg:

Kraftpaket: kann sie vielleicht mit einem Schlag schaffen.

Schwache Figur: braucht zwei Schläge, kann dafür sehr genau vor einem Hindernis liegen bleiben.

Eine enge Bahn mit vielen Banden:

Kraftpaket: gefährlich, weil ein kleiner Fehler viel Geschwindigkeit erzeugt.

Feinfühlige Figur: kann sehr exakt dosieren.

Eine Windmühlenbahn:

Hier ist möglicherweise überhaupt nicht die maximale Kraft wichtig, sondern das **Timing und die Zielgenauigkeit**.

Damit bekommt eine Bahn je nach Charakter eine andere optimale Strategie. Und genau dadurch helfen vier Figuren auch gegen das ursprüngliche Problem der wenigen Bahnen: **36 Bahnen × vier unterschiedlich spielende Charaktere fühlen sich schon wesentlich weniger wie nur 36 Herausforderungen an.**

Ich würde Kraft nicht einfach linear machen

Das Kraftpaket sollte nicht nur einen längeren Balken haben. Viel interessanter wäre:

Schwache Figur

0-----100



langsamer Cursor

Maximum Ballgeschwindigkeit: 75 %

Pete

0-----100



normaler Cursor

Maximum Ballgeschwindigkeit: 100 %

Kraftpaket

0-----100



schneller Cursor

Maximum Ballgeschwindigkeit: 130 %

Der sichtbare Balken bleibt also immer gleich groß. Der Spieler muss nicht plötzlich mit verschiedenen UI-Skalen umgehen. **Die Figur verändert die Physik hinter dem Balken und dessen Geschwindigkeit.**

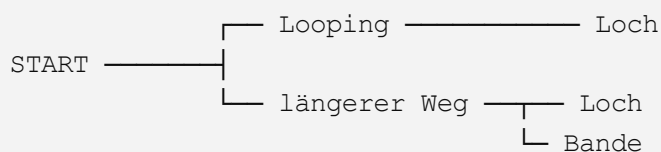
Das ist wesentlich intuitiver.

Noch wichtiger: keine Figur darf Bahnen unspielbar machen

Das wäre eine meiner zentralen Regeln beim Leveldesign.

Wenn beispielsweise eine Looping-Brücke mindestens **80 Kraft** benötigt und die schwache Figur maximal **75** erreicht, wäre sie schlicht ausgeschlossen.

Stattdessen könnte eine solche Bahn beispielsweise zwei Wege bieten:



Das Kraftpaket kann den riskanten Looping nehmen.

Die schwächere Figur nimmt den längeren technischen Weg.

Und Pete kann beides versuchen.

Das wäre richtig gutes Charakterdesign, weil die Figuren nicht nur Zahlen verändern, sondern gelegentlich sogar die Entscheidung auf einer Bahn.

Man könnte daraus sogar vier klare Persönlichkeiten machen

Ich würde sie optisch ebenfalls sofort verständlich gestalten:

Pete – der Allrounder

klassischer Minigolfer, dicht am Original. Alles Mittelmaß.

Mia – die Feinfühlige

kleine oder schwächliche Figur. Langsame Kraftanzeige, geringe Maximalstärke. Sehr angenehm bei kurzen technischen Bahnen.

Bruno – das Kraftpaket

groß, kräftig, vielleicht etwas übermotiviert. Hohe Schlagkraft, Kraftanzeige rast schneller. Nach einem zu starken Schlag könnte er erschrocken dem Ball hinterherschauen. 😊

Professor / Techniker

vielleicht ältere Figur mit Brille. Sehr kontrollierte Ballabweichung, aber winziges perfektes Timingfenster.

Und Mini-Putt lebt ja gerade von den kleinen Animationen der Spielfigur. Deshalb könnte jede Figur auch eigene Reaktionen haben.

Bruno zerbricht nach einem schlechten Schlag vielleicht nicht den Schläger – er verbiegt ihn versehentlich.

Die Feinfühlige jubelt über einen perfekten technischen Bandenschlag.

Der Professor schaut nach einem Fehlschlag auf seinen Schläger, als müsse die Physik daran schuld sein.

Das kostet programmtechnisch kaum etwas, bringt aber enorm viel Persönlichkeit.

Ich würde sogar noch einen Schritt weitergehen: **Charaktere sollten keine freischaltbaren Verbesserungen bekommen.** Keine `+5 % Kraft`, keine besseren Schläger usw. Dann bleibt das Spiel ein Geschicklichkeitsspiel und wird nicht zu einem Progressionssystem.

Der Fortschritt sollte stattdessen aus **neuen Kursen, schwierigeren Bahnen, kosmetischen Dingen und vielleicht neuen Figuren mit anderen Spielstilen** bestehen.

Und damit hätten wir schon drei sehr starke Säulen für deine Neuinterpretation:

Mini-Putt-Originalmechanik + unterschiedliche Golfer + sehr viele neue Bahnen.

Als Nächstes würde ich tatsächlich das **Bahnsystem** entwerfen: Welche Bauteile brauchen wir, damit man daraus vielleicht 100–200 unterschiedliche Löcher bauen kann, ohne jedes Loch programmieren zu müssen. Das ist wahrscheinlich der entscheidende Punkt für dieses Projekt.

ja das klingt sinnvoll.

Dann würde ich das Projekt jetzt wirklich **von den Bahnen her denken**. Wenn das System einmal sauber modular aufgebaut ist, sollten neue Löcher später hauptsächlich Leveldesign und kaum Programmierung bedeuten.

Der Grundgedanke wäre: Eine Bahn besteht nicht aus einem speziell programmierten Level, sondern aus einer Kombination wiederverwendbarer Bausteine.

Baustein	Funktion
Grünfläche	normale Ballbewegung
Bande	Reflexion
Schräge	konstante Kraft auf den Ball
Rough/Sand	stärkere Reibung
Wasser	Strafschlag + Rücksetzen
Loch	Ziel
Rampe	Geschwindigkeits-/Richtungsänderung
Brücke	überquert andere Bereiche
Tor	öffnet/schließt
Windmühle	rotierendes Kollisionsobjekt
Pendel	zeitabhängiges Hindernis
Förderband	bewegt Ball zusätzlich
Teleporter/Röhre	Eingang → Ausgang
Kanone	nimmt Ball auf und schießt ihn weiter
Schalter	verändert andere Objekte

Damit könnte bereits sehr viel entstehen.

Interessant finde ich besonders, die Bahnen in **drei Ebenen von Komplexität** einzuteilen.

Klasse A – Golfbahnen: Hier entscheidet fast nur Geometrie. Bandenschüsse, Winkel, Gefälle und Kraft. Diese Bahnen wären dem Traditional-Kurs am ähnlichsten.

Klasse B – Mechanische Bahnen: Windmühlen, Tore, Brücken, Pendel und bewegliche Plattformen. Hier kommt Timing dazu.

Klasse C – Abenteuerbahnen: Der Ball wird plötzlich Teil einer kleinen Maschine: Kanone, Röhre, Aufzug, Schalter, Wasserlauf usw. Das entspricht eher dem verrückten Classic-Kurs.

Dadurch könnten wir beispielsweise Kurse bewusst mischen:

```
3 technische + 3 mechanische + 3 Abenteuerbahnen
```

oder ein Thema komplett durchziehen.

Ein ganz wichtiger Baustein wären Zonen

Viele Effekte müssen gar keine komplizierten Objekte sein. Eine unsichtbare oder sichtbare Zone reicht:

```
Zone
├─ Reibung
├─ Beschleunigung X/Y
├─ Maximalgeschwindigkeit
├─ Ball stoppen?
├─ Strafschlag?
├─ Teleportziel?
└─ Sound/Animation
```

Damit bekommt man erstaunlich viel.

Eine Schräge wäre einfach:

```
Beschleunigung = Richtung des Gefälles
```

Sand:

```
Reibung = hoch
```

Eis:

```
Reibung = sehr niedrig
```

Förderband:

```
Beschleunigung = Richtung des Bandes
```

Wasser:

```
Ball stoppen + Strafschlag + zurücksetzen
```

Dadurch braucht man für viele Bahnideen überhaupt keinen Spezialcode.

Dann die beweglichen Hindernisse

Auch die würde ich auf wenige Grundbewegungen reduzieren:

```
Rotation  
Pendelbewegung  
Hin-und-Her  
Kreisbahn  
Auf/Zu  
Ein/Aus
```

Eine Windmühle ist dann:

```
Rotation + Kollisionsfläche
```

ein Tor:

```
Auf/Zu + Kollisionsfläche
```

ein fahrender Block:

```
Hin-und-Her + Kollisionsfläche
```

und ein Hammer:

```
Pendelbewegung + Kollisionsfläche
```

Der Leveldesigner wählt nur Geschwindigkeit, Startposition, Winkel und Zeitablauf.

Das ist wichtig, weil wir sonst bei 100 Bahnen irgendwann 80 verschiedene Spezialobjekte programmieren.

Für außergewöhnliche Dinge brauchen wir Trigger

Das wäre wahrscheinlich das mächtigste System.

Beispielsweise:

```
Ball berührt Schalter A  
↓  
Tor B öffnet  
↓  
Brücke C fährt aus  
↓  
Windmühle D wird langsamer
```

Oder:

```
Ball fährt in Kanone
    ↓
Ball einfrieren
    ↓
Kanonenanimation
    ↓
0,8 Sekunden warten
    ↓
Ball Position = Ausgang
Ball Geschwindigkeit = 450
Ball Richtung = 35°
```

Damit kann man später regelrechte kleine Minigolf-Maschinen bauen.

Und jetzt kommt etwas, das für deine vier Figuren interessant wird

Jede Bahn könnte beim Erstellen automatisch geprüft werden.

Zum Beispiel:

```
Pete:
MaxSpeed = 100

Schwache Figur:
MaxSpeed = 75

Kraftpaket:
MaxSpeed = 130
```

Der Editor könnte einen Testmodus besitzen:

„Ist dieses Loch mit allen Figuren grundsätzlich erreichbar?“

Damit verhindert man versehentlich unmögliche Bahnen.

Noch besser wäre eine Anzeige:

```
Erforderliche Mindestgeschwindigkeit:

Hauptweg:      62
Looping:       105
Sprungrampe:   118
```

Dann sieht der Designer sofort:

- schwache Figur → Hauptweg
- Pete → Hauptweg
- Kraftpaket → alle drei Möglichkeiten

Genau dadurch werden die Charaktere richtig interessant.

Ich würde außerdem Bahn und Dekoration strikt trennen

Beispielsweise kann dieselbe Grundbahn so aussehen:

Pirateninsel

Kanone, Holzbrücke, Wasser, Palmen.

Oder:

Steampunk-Fabrik

Dampfkanone, Metallbrücke, Säurebecken, Zahnräder.

Oder:

Zauberwald

magisches Portal, Baumwurzelbrücke, Teich, drehende Pilze.

Spielmechanisch kann darunter exakt dieselbe Logik liegen.

Das bedeutet, wir könnten aus vielleicht **20–30 technischen Elementen eine sehr große Zahl unterschiedlicher Bahnen erzeugen**, ohne ständig neue Mechaniken schreiben zu müssen.

Und ich würde noch etwas vom Original übernehmen: Eine Bahn sollte meistens **auf einen Blick verständlich sein**. Der Spieler sieht sie und denkt:

„Aha — wenn ich dort an die Bande spiele, komme ich vielleicht hinter der Windmühle vorbei.“

Nicht erst fünf Menüs öffnen und Statistiken analysieren.

Für unser nächstes konkretes Designstück würde ich deshalb einen **Bahn-Baukasten V1 mit etwa 20 Elementen** definieren. Damit könnten wir anschließend probeweise zehn ganz unterschiedliche Löcher auf Papier konstruieren und schauen, ob uns noch wichtige Bausteine fehlen. Das wäre schon fast die Grundlage für einen späteren Level-Editor.